



Clase 8

Aseguramiento de Calidad

Andres Caicedo · Hardcore AI | 30X · 2h · 120 min

¿Dónde estamos en el journey?

Apertura y Contexto

- ▶ **Clases 4-5** → Inception + Construction con AI-DLC (specs, ADRs, infra design)
- ▶ **Clase 6** → Scaffolding e implementación con arneses (skills, `PRODUCT.md`, `DESIGN.md`)
- ▶ **Clase 7** → Orquestación AI-DLC → Linear, code review automatizado y memoria de proyecto
- ▶ **Hoy** → El filtro de calidad antes de producción: pruebas funcionales

💡 El producto existe y está orquestado. La pregunta ahora es: ¿cómo garantizas que funciona como el usuario espera?

El costo de no probar

¿Por qué las pruebas funcionales son el último filtro?

- ▶ Un bug en producción cuesta **10x más** que uno detectado en testing
- ▶ Sin cobertura funcional: el usuario es tu tester involuntario
- ▶ **Tres capas sin cobertura** → problemas distintos:
 - ▶ Sin E2E web: flujos de usuario rotos en browser
 - ▶ Sin pruebas de API: contratos rotos, endpoints inconsistentes
 - ▶ Sin evaluación de agentes: respuestas incorrectas o dañinas llegan al usuario
- ▶ Con IA: lo que antes tomaba días de QA manual, hoy se hace en horas

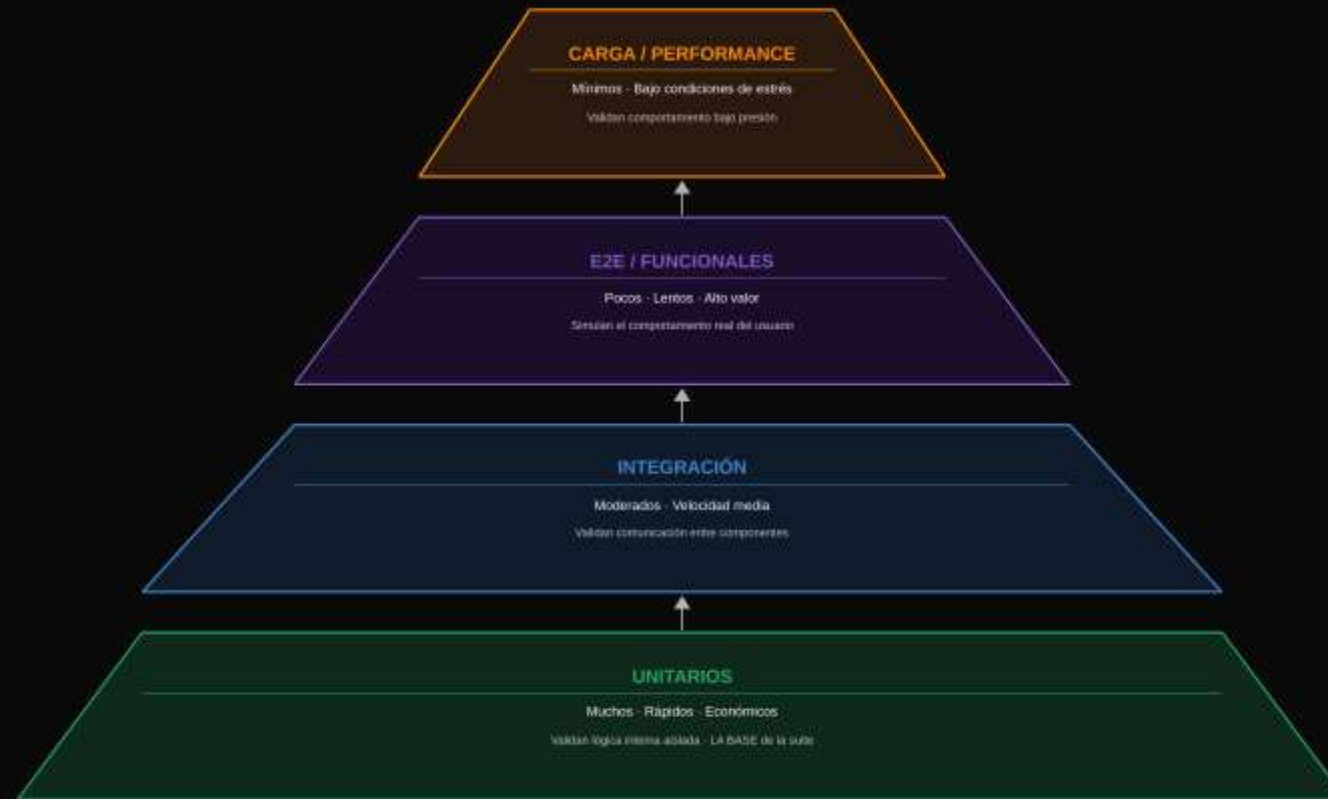
02 — SECCIÓN

La Pirámide de Pruebas

¿Dónde invertir el esfuerzo de calidad?

La Pirámide de Pruebas

Del mayor volumen al mayor valor



Dónde invertir el esfuerzo

Trade-offs y el anti-patrón del Ice Cream Cone

- ▶ **Base amplia de unitarios:** rápidos, baratos, dan feedback inmediato
- ▶ **Capa media de integración:** validan que los componentes se comunican
- ▶ **Punta de E2E:** pocos pero de alto valor — los que el usuario siente
- ▶ **Anti-patrón "Ice Cream Cone":** invertir la pirámide — todo E2E, pocos unitarios → frágil y lento

💡 En este curso: unitarios e integración se cubren con tu pipeline de Construction (clases 4-7) · E2E + API + Agentes → hoy

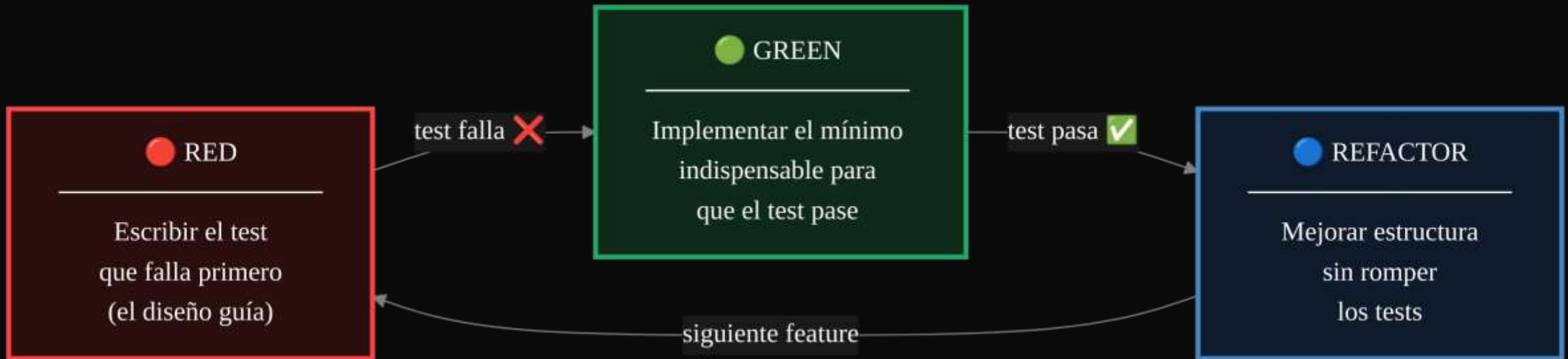
03 — SECCIÓN

BDD, TDD y Gherkin

Comportamiento como especificación ejecutable

TDD: Red → Green → Refactor

El ciclo que guía el diseño, no solo la verificación



💡 TDD no es sobre tests — es sobre diseño. El test es la especificación de lo que el código debe hacer.

BDD: Comportamiento como especificación

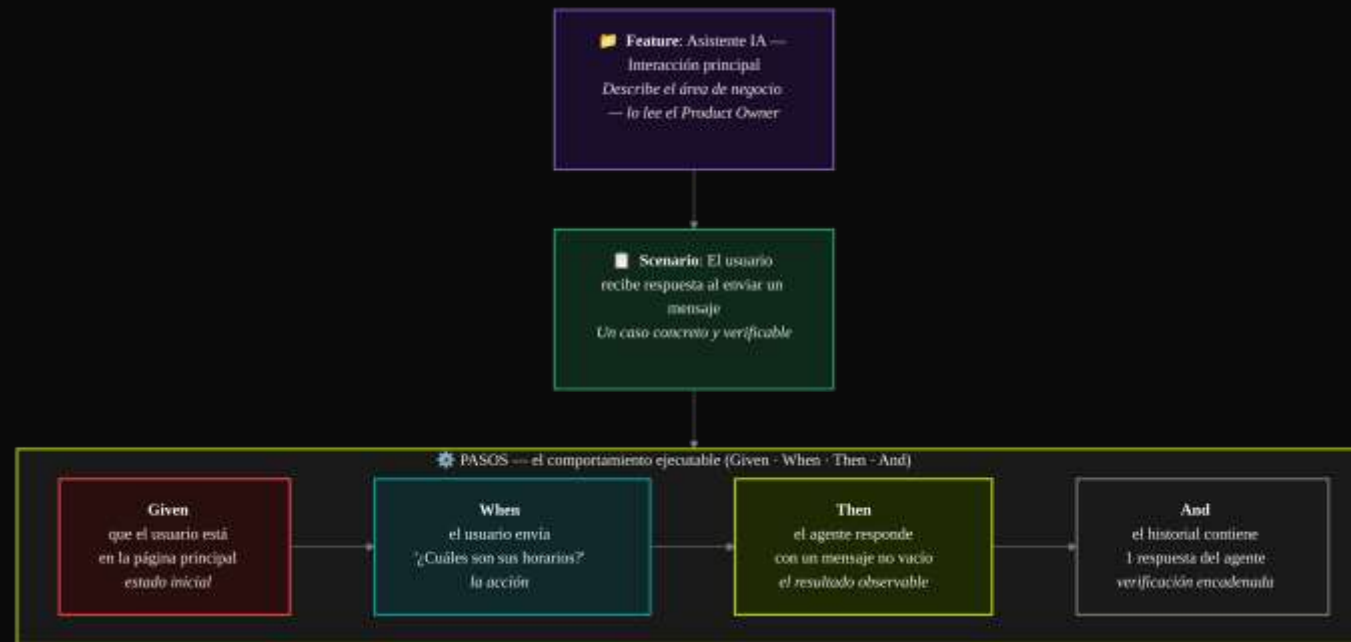
El lenguaje que comparten negocio y desarrollo



💡 BDD aplica a flujos web, contratos de API y conversaciones de agentes. El negocio puede leer y validar los escenarios.

Gherkin: El .feature file

Anatomía de un escenario ejecutable

[► DEMO](#)

Abrir `tests/features/asistente.feature` → mostrar escenario Gherkin → abrir `steps/asistente.steps.ts` → ejecutar `npx playwright test tests/bdd/` → revisar reporte HTML

04 — SECCIÓN

Patrones de Diseño para Testing

Cómo escalar sin que el mantenimiento te consuma

El problema sin patrones

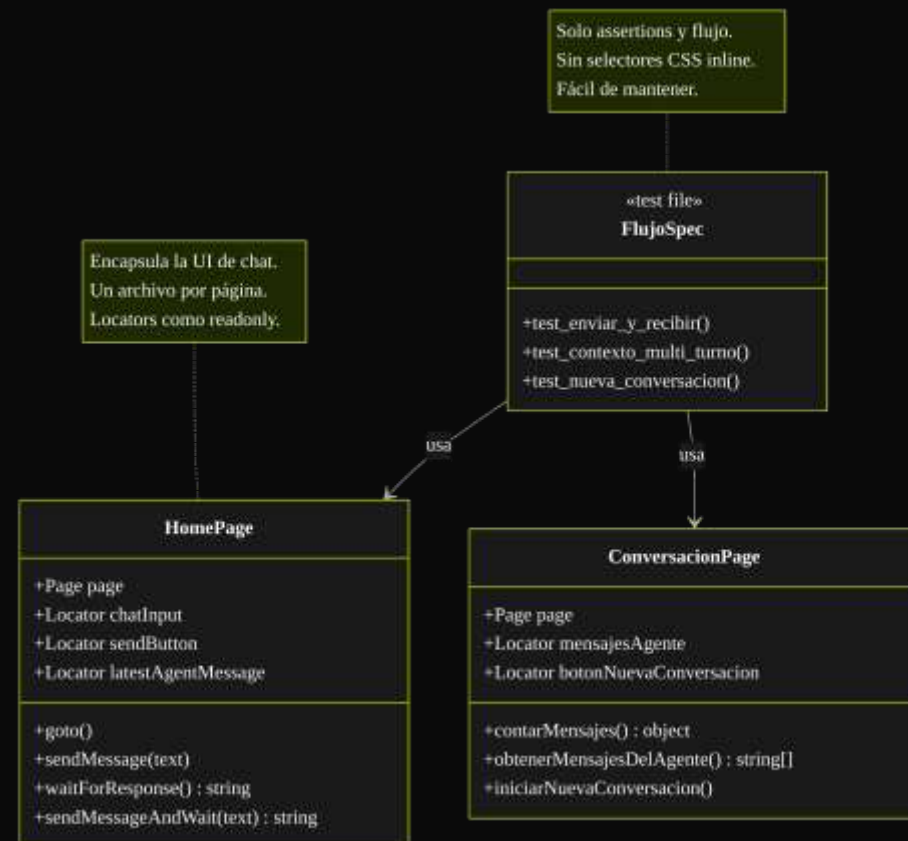
¿Qué pasa cuando los tests crecen sin estructura?

- ▶ **Sin POM:** los selectores CSS están duplicados en 40 archivos
- ▶ Un cambio en la UI → rompe 40 tests → horas de mantenimiento
- ▶ Tests con lógica de navegación mezclada con assertions → ilegibles
- ▶ El equipo deja de actualizar los tests porque "es muy caro"

💡 Un mismo flujo, implementado sin patrón vs. con POM: misma funcionalidad, diferente costo de mantenimiento.

Page Object Model (POM)

Encapsular la UI en clases reutilizables



Screenplay Pattern y Patrones de API

Más allá de POM para flujos complejos

- ▶ **Screenplay Pattern:**
- ▶ Modela tests como **actores** que realizan **tareas**
- ▶ Ventaja sobre POM: flujos multi-actor y multi-rol son más naturales
- ▶ Ejemplo: ``Carlos.attemptsTo(EnviarMensaje.con("Hola"), VerificarRespuesta.noVacia())``
- ▶ **Patrones para API Testing:**
- ▶ **Request Builders:** encapsulan construcción de requests
- ▶ **Response Validators:** validan estructura de respuesta
- ▶ **Contract Testing:** verifican que el contrato del endpoint no cambia

💡 Ver ``fixtures/request-builders.ts`` en el repo ``qa-api`` — el mismo patrón en acción

05 — SECCIÓN

Pruebas de Agentes Inteligentes

El patrón Persona + Juez

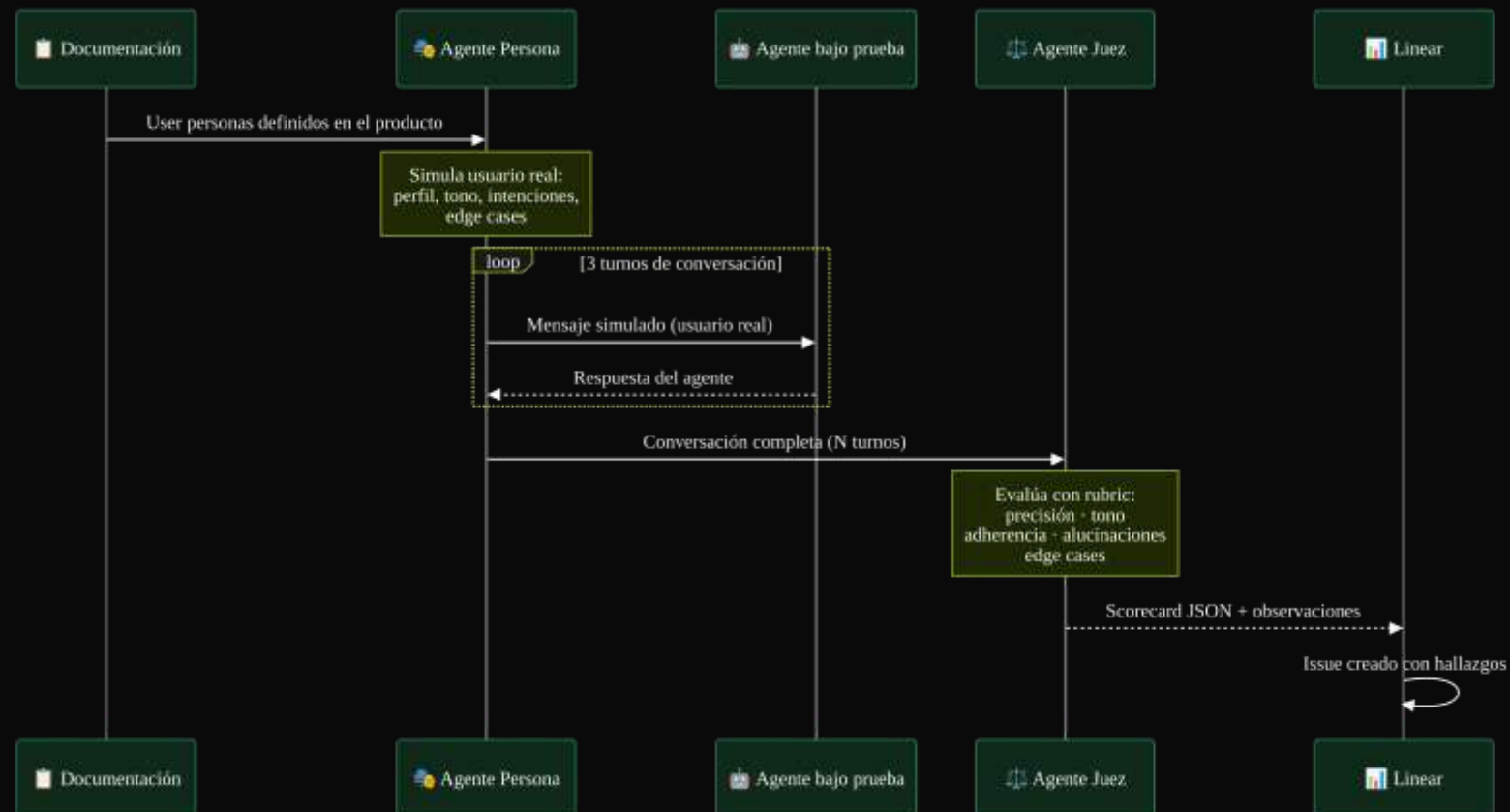
El desafío: respuestas no determinísticas

¿Cómo verificar algo que no siempre responde igual?

- ▶ Los agentes de IA producen **respuestas distintas** cada vez
- ▶ No se puede comparar con ``expect(respuesta).toBe("texto exacto")``
- ▶ Se necesita evaluación **semántica**: ¿es relevante? ¿es correcta? ¿es segura?
- ▶ Un test binario (pass/fail) no captura la calidad real del agente
- ▶ **Dimensiones que importan:**
 - ▶ Precisión factual · Relevancia · Tono · Adherencia al system prompt
 - ▶ Manejo de edge cases · Ausencia de alucinaciones

Patrón Persona + Juez

Evaluar agentes con agentes



Golden Datasets

La base de referencia para calibrar el Juez



► DEMO

Ejecutar `npm run eval` en `qa-agent-eval` → ver conversación generada → ver scorecard con puntuaciones → mostrar issue creado en Linear

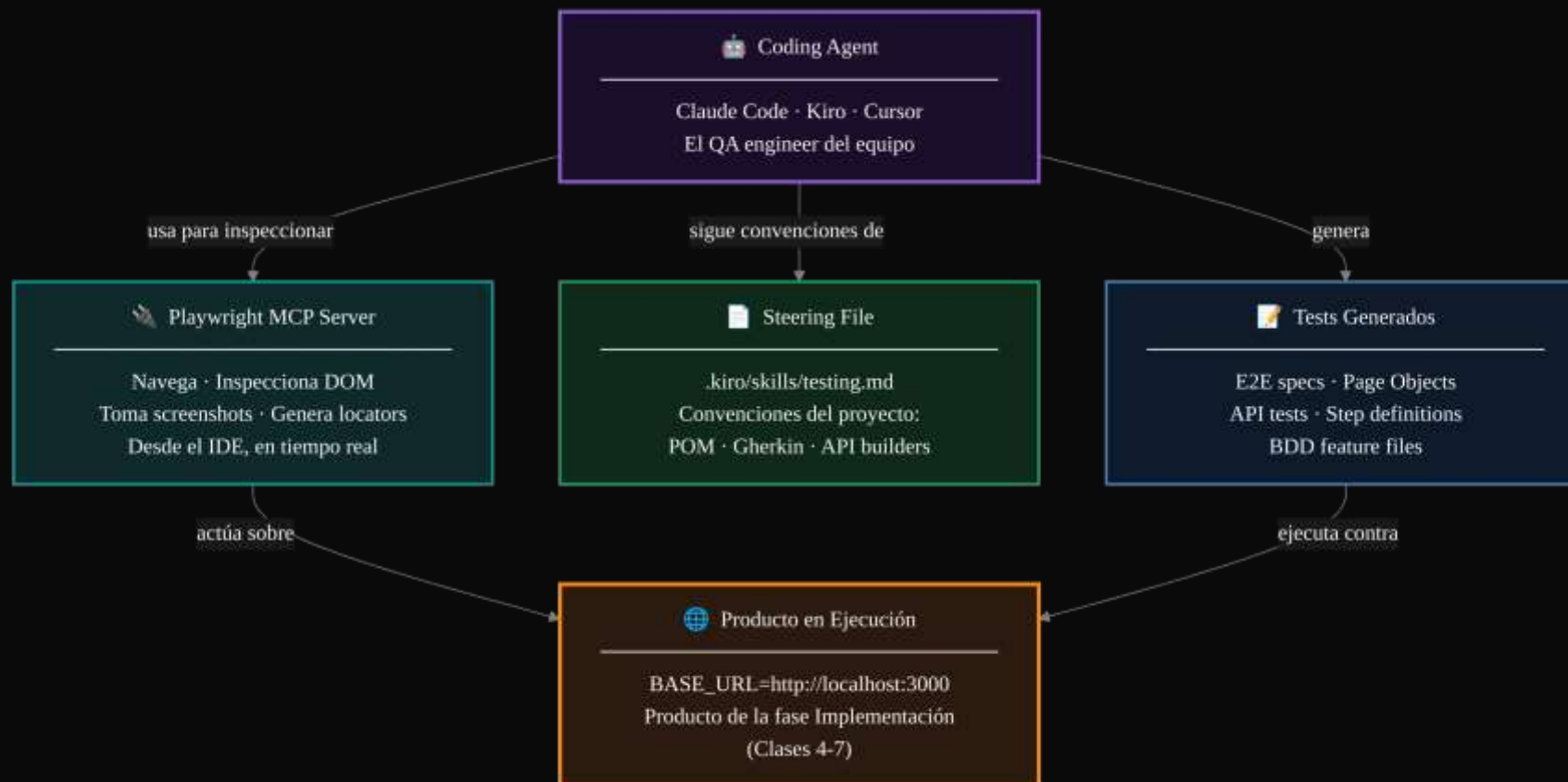
06 — SECCIÓN

El Stack de IA para Testing

Show & Tell en repositorio real

El Stack: Coding Agent + MCP + Steering File

Las tres piezas que lo hacen posible



El Steering File para QA

Las convenciones del proyecto en manos del agente

- ▶ Qué incluye `.kiro/skills/testing.md`:
- ▶ Estructura de carpetas (``tests/e2e/``, ``pages/``, ``steps/``, ``tests/features/``)
- ▶ Convenciones de naming (POM, specs, step definitions, features)
- ▶ Patrones obligatorios: POM primero, luego tests que lo consumen
- ▶ Configuración Playwright: ``trace``, ``screenshot``, ``baseURL``
- ▶ Instrucciones específicas al agente: usar MCP antes de generar locators

TRANSICIÓN

Kahoot

Validar comprensión antes del hands-on

- ▶ Pirámide de pruebas: niveles y trade-offs
- ▶ BDD vs TDD: cuándo usar cada uno
- ▶ Gherkin · POM · Screenplay
- ▶ Patrón Persona + Juez · Golden datasets

Para la próxima sesión

Cobertura funcional completa de tu producto

- ▶ ☐ **Steering file de testing** — ``.kiro/skills/testing.md`` o equivalente con convenciones de POM, Gherkin y API builders
- ▶ ☐ **E2E con POM** — al menos **2 flujos críticos** del producto cubiertos con Page Objects + tests Playwright
- ▶ ☐ **Tests de API** — al menos **1 endpoint clave** con request builders + contract tests
- ▶ ☐ **Evaluación de agentes** (si aplica) — golden dataset + rubric + ejecutar Persona + Juez al menos 1 vez

💡 El objetivo es construir el **stack de QA personal** que vas a reusar en todos tus proyectos. Los tests son la red de seguridad antes de producción.